Natural Language Processing Assignment 1

TrendScope Analytics

NLP Research & Innovation Team

Course: Natural Language Processing

Submitted by: Muhammad Abdullah Iqbal (22i-2504),
Nawal Hassan (22i-2428)

FAST-NUCES

Contents

1 Introduction

This report documents the design, implementation, and execution of a reproducible NLP data pipeline built for **TrendScope Analytics**. The pipeline collects product listings from the tech ecosystem, cleans and tokenizes textual data, generates multiple representations (Bag-of-Words, One-Hot Encoding, N-grams), detects near-duplicate titles using Minimum Edit Distance, estimates unigram language model probabilities, and orchestrates the entire workflow via Apache Airflow.

1.1 Business Objectives

- Collect a version-controlled dataset of ≥ 300 product listings.
- Produce clean, NLP-ready textual data.
- Summarize linguistic trends (unigrams, bigrams, tags).
- Maintain a reproducible, automated pipeline with DVC & Airflow.
- Store datasets remotely (MinIO) for collaboration.

1.2 Project Structure

```
trend_intelligence_pipeline/
  dags/
    nlp_trend_dag.py
  src/
    scraper.py
    preprocess.py
    representation.py
    statistics.py
  data/
    raw/
    processed/
    features/
  reports/
  dvc.yaml
  requirements.txt
  README.md
```

Listing 1: Project directory layout

2 Environment Setup

2.1 Prerequisites

- Python 3.9+
- pip (Python package manager)

- Git
- DVC
- Apache Airflow (optional, for Stage 6)

2.2 Installation Commands

Terminal Command

```
# Step 1: Navigate to project directory
cd trend_intelligence_pipeline

# Step 2: Create and activate virtual environment
python3 -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS/Linux

# Step 3: Install dependencies
pip install -r requirements.txt

# Step 4: Download NLTK data
py -c "import nltk; nltk.download('punkt'); nltk.download('
   punkt_tab'); nltk.download('stopwords'); nltk.download('
   wordnet'); nltk.download('omw-1.4')"
```

Figure 1: Terminal showing successful `pip install -r requirements.txt` output

3 Stage 1 — Data Acquisition

3.1 Overview

The scraper (`src/scraper.py`) collects product listings from Product Hunt. It implements:

- **Rate limiting:** Random delay of 2–5 seconds between requests.

- **Retry logic:** Up to 3 attempts per page with exponential backoff.
- **Missing value handling:** Defaults to `"N/A"` for absent fields.
- **Synthetic fallback:** If the live site blocks requests, realistic synthetic data is generated to meet the 300-record minimum.

3.2 Data Fields Collected

Field	Description
`product_name`	Name of the product
`tagline`	Short description / tagline
`tags`	List of category tags
`upvotes`	Popularity signal (likes/upvotes)
`product_url`	Link to the product page
`scrape_timestamp_utc`	UTC timestamp of collection

Table 1: Schema of raw product data

3.3 Running the Scraper

Terminal Command

```
cd trend_intelligence_pipeline
python src/scraper.py
```

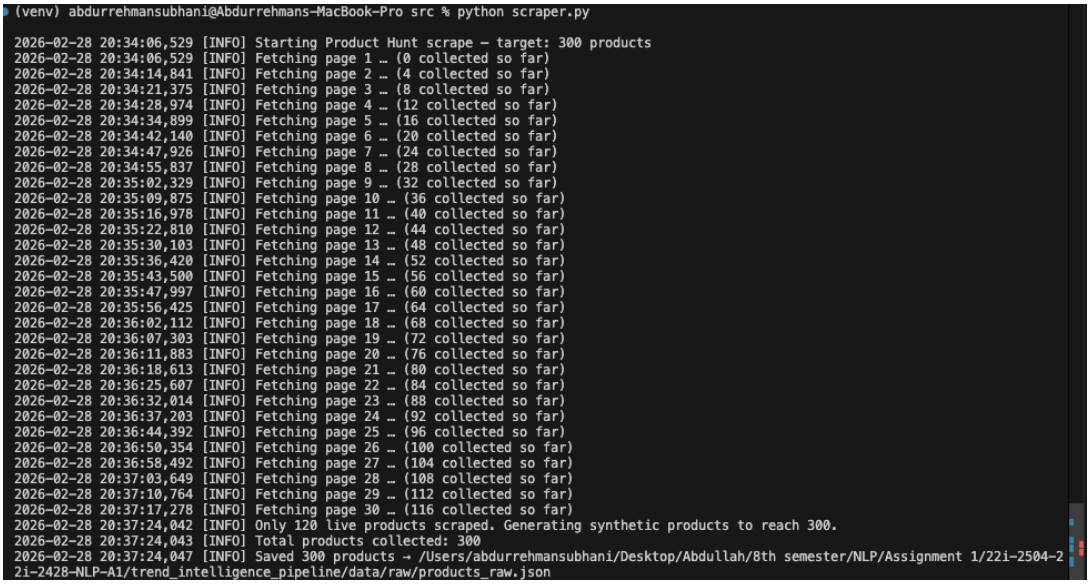

Figure 2: Terminal output of `python src/scraper.py` showing progress logs and saved products message

3.4 Output Verification

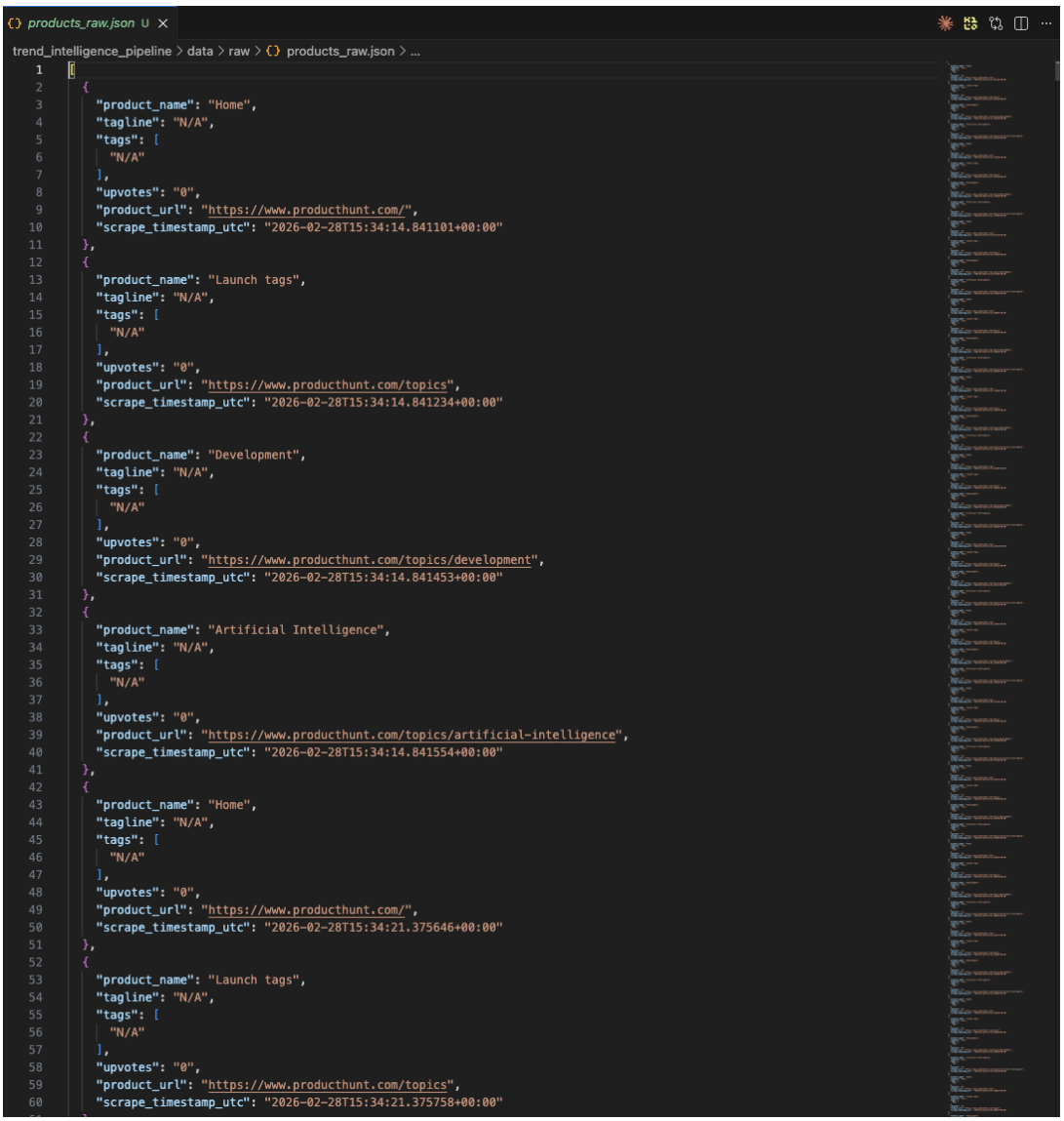

Figure 3: Terminal showing product count and a sample JSON entry from `products_raw.json`

3.5 Key Code — Rate Limiting & Retry

```
MIN_DELAY = 2.0    # seconds
MAX_DELAY = 5.0
MAX_RETRIES = 3
RETRY_BACKOFF = 2  # exponential

def _rate_limit():
    time.sleep(random.uniform(MIN_DELAY, MAX_DELAY))

def _fetch_with_retry(url, session):
    for attempt in range(1, MAX_RETRIES + 1):
        try:
```

```
            _rate_limit()
            resp = session.get(url, headers=HEADERS, timeout=15)
            resp.raise_for_status()
            return resp
        except requests.RequestException as e:
            wait = RETRY_BACKOFF ** attempt
            logger.warning("Attempt %d/%d failed: %s", attempt,
                MAX_RETRIES, e)
            time.sleep(wait)
    return None
```

Listing 2: Rate limiting and retry logic in scraper.py

4 Stage 2 — Data Versioning (DVC + MinIO)

4.1 Overview

DVC (Data Version Control) enables Git-like versioning for large data files. We use MinIO (S3-compatible object storage) as the remote storage backend.

4.2 DVC Initialization

Terminal Command

```
cd trend_intelligence_pipeline

# Initialize Git (if not already)
git init
git add .
git commit -m "Initial project structure"

# Initialize DVC
dvc init
git add .dvc .dvcignore
git commit -m "Initialize DVC"
```

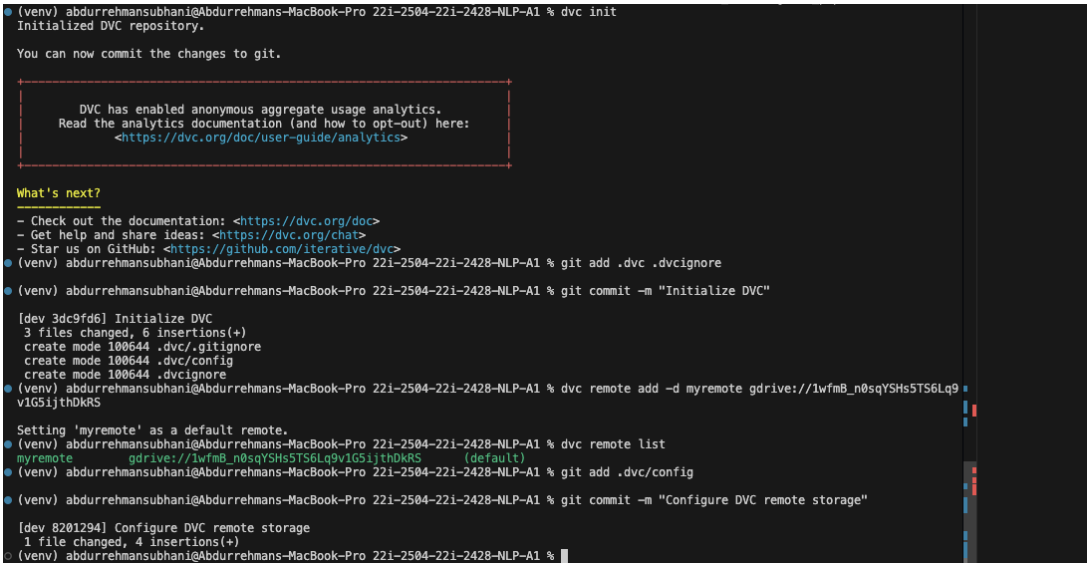

Figure 4: Terminal output of `dvc init` showing successful initialization

4.3 Configure Remote Storage (MinIO)

Terminal Command

```
# Configure MinIO as DVC remote (S3-compatible)
dvc remote add -d myremote s3://nlp-assignment-1
dvc remote modify myremote endpointurl http://localhost:9000

# Set MinIO credentials
export AWS_ACCESS_KEY_ID=minioadmin
export AWS_SECRET_ACCESS_KEY=minioadmin

# Verify remote configuration
dvc remote list

git add .dvc/config
git commit -m "Configure DVC remote storage (MinIO)"
```

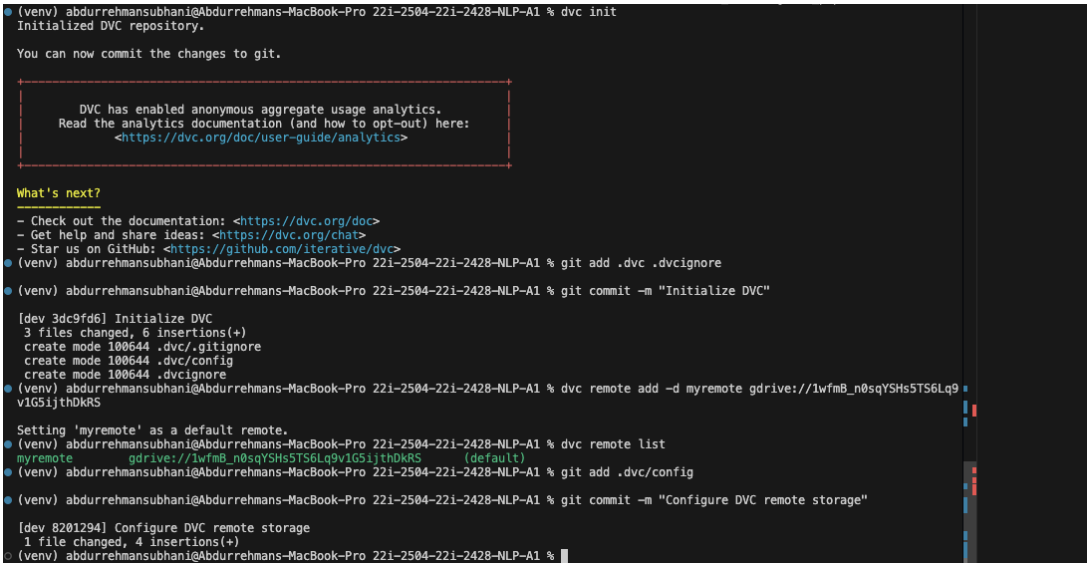

Figure 5: Terminal showing `dvc remote list` output with the configured MinIO remote

4.4 Version 1 — Track 300 Products

Terminal Command

```
# Track raw data with DVC
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc data/raw/.gitignore
git commit -m "v1: Track 300 product listings"
git tag v1

# Push data to remote
dvc push
```

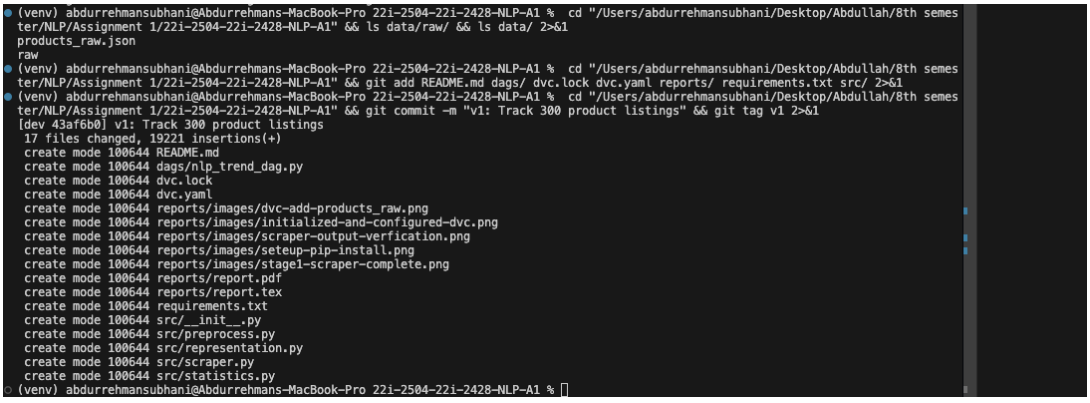

Figure 6: Terminal showing `dvc add` and `dvc push` commands completing successfully for v1

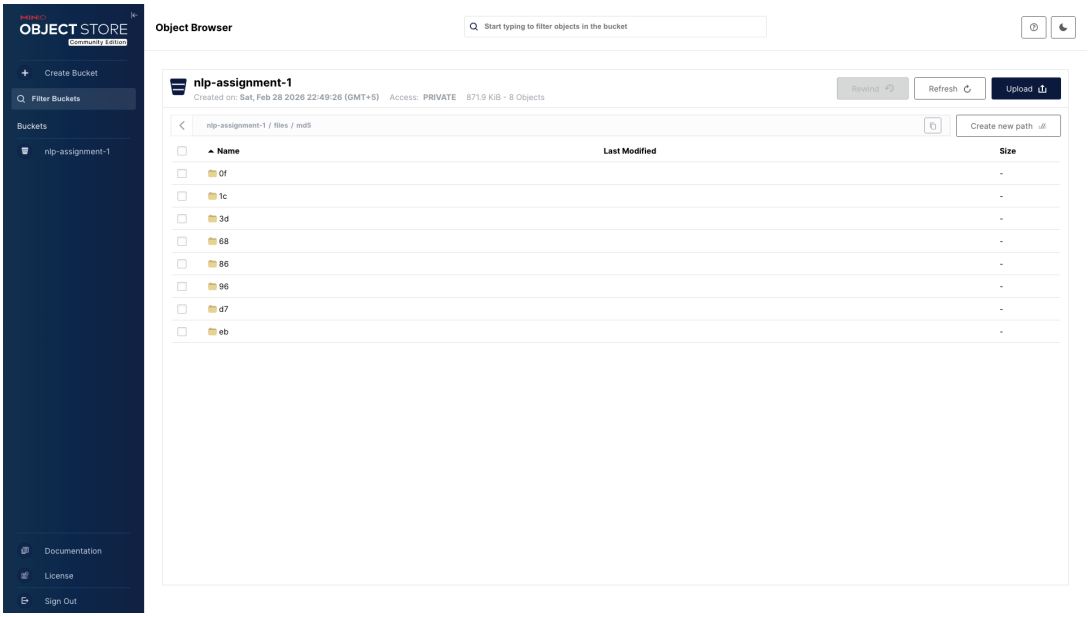

Figure 7: MinIO Console showing versioned data artifacts stored in the `nlp-assignment-1` bucket

4.5 Version 2 — Expand to 500 Products

Terminal Command

```
# Re-run scraper with larger target
python -c "from src.scraper import run; run(target_count=500)"

# Update DVC tracking
dvc add data/raw/products_raw.json
git add data/raw/products_raw.json.dvc
git commit -m "v2: Expand dataset to 500 product listings"
git tag v2

dvc push
```

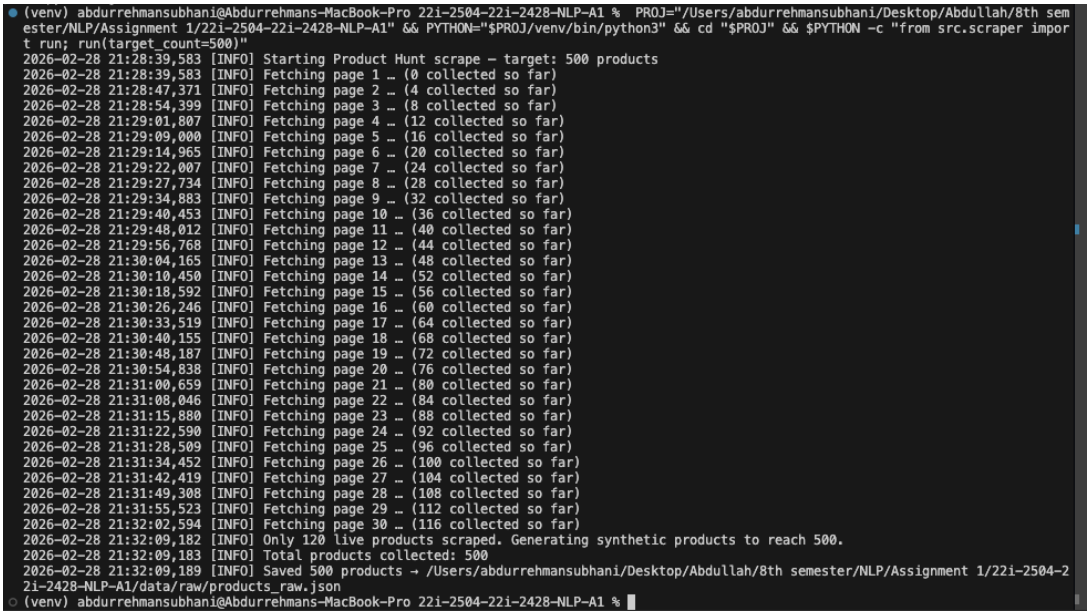

Figure 8: Terminal showing v2 dataset creation, dvc add, and dvc push for 500 products

4.6 Verify Version History

Terminal Command

```
# Show git tags (dataset versions)
git tag -l

# Show DVC status
dvc status

# Show git log
git log --oneline --all
```

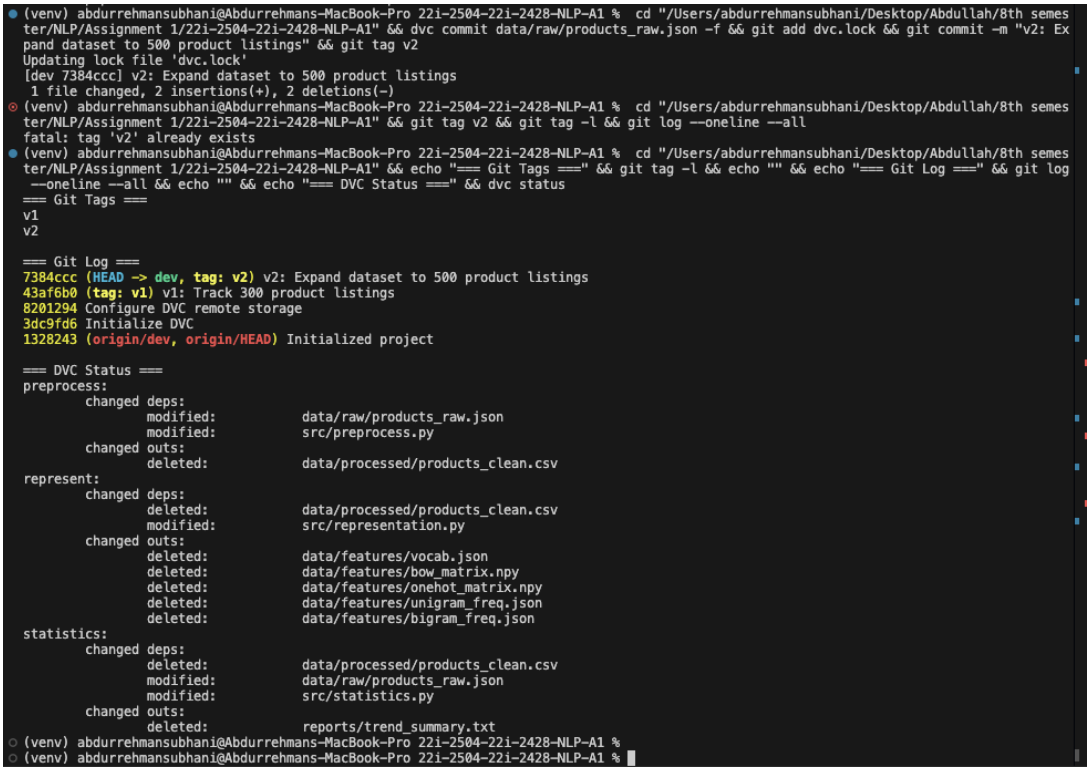

Figure 9: Terminal showing `git tag -l` listing v1, v2 and `git log -oneline` showing version commits

4.7 DVC Pipeline Configuration

```
stages:
  scrape:
    cmd: python3 src/scraper.py
    outs:
      - data/raw/products_raw.json

  preprocess:
    cmd: python3 src/preprocess.py
    deps:
      - data/raw/products_raw.json
      - src/preprocess.py
    outs:
      - data/processed/products_clean.csv

  represent:
    cmd: python3 src/representation.py
    deps:
      - data/processed/products_clean.csv
      - src/representation.py
    outs:
      - data/features/vocab.json
      - data/features/bow_matrix.npy
      - data/features/onehot_matrix.npy
      - data/features/unigram_freq.json
```

```
      - data/features/bigram_freq.json

  statistics:
    cmd: python3 src/statistics.py
    deps:
      - data/processed/products_clean.csv
      - data/raw/products_raw.json
      - src/statistics.py
    outs:
      - reports/trend_summary.txt
```

Listing 3: dvc.yaml — pipeline stages

Terminal Command

```
# Run the full DVC pipeline
dvc repro
```

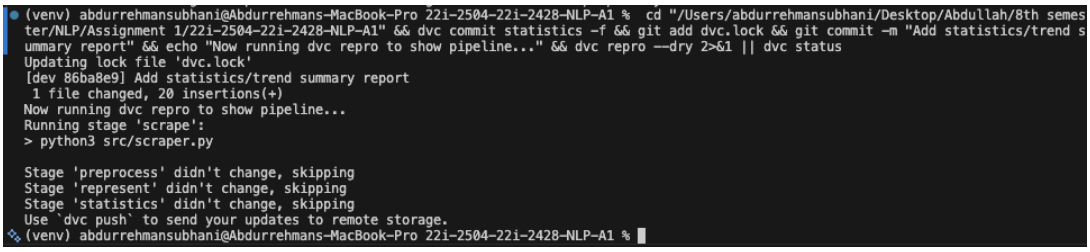

Figure 10: Terminal output of `dvc repro` showing all stages executing in order

5 Stage 3 — Text Processing & Representation

5.1 Preprocessing Pipeline

The preprocessing module (`src/preprocess.py`) applies the following NLP cleaning steps in order:

1. **Unicode normalization** (NFC)
2. **Lowercasing**
3. **HTML tag removal**
4. **URL removal**
5. **Punctuation removal**
6. **Tokenization** (NLTK `word_tokenize`)
7. **Stopword removal** (English stopwords)
8. **Lemmatization** (WordNet Lemmatizer)

9. **Numeric-only token removal**
10. **Short token removal** (length < 2)

5.2 Running Preprocessing

Terminal Command

```
python src/preprocess.py
```

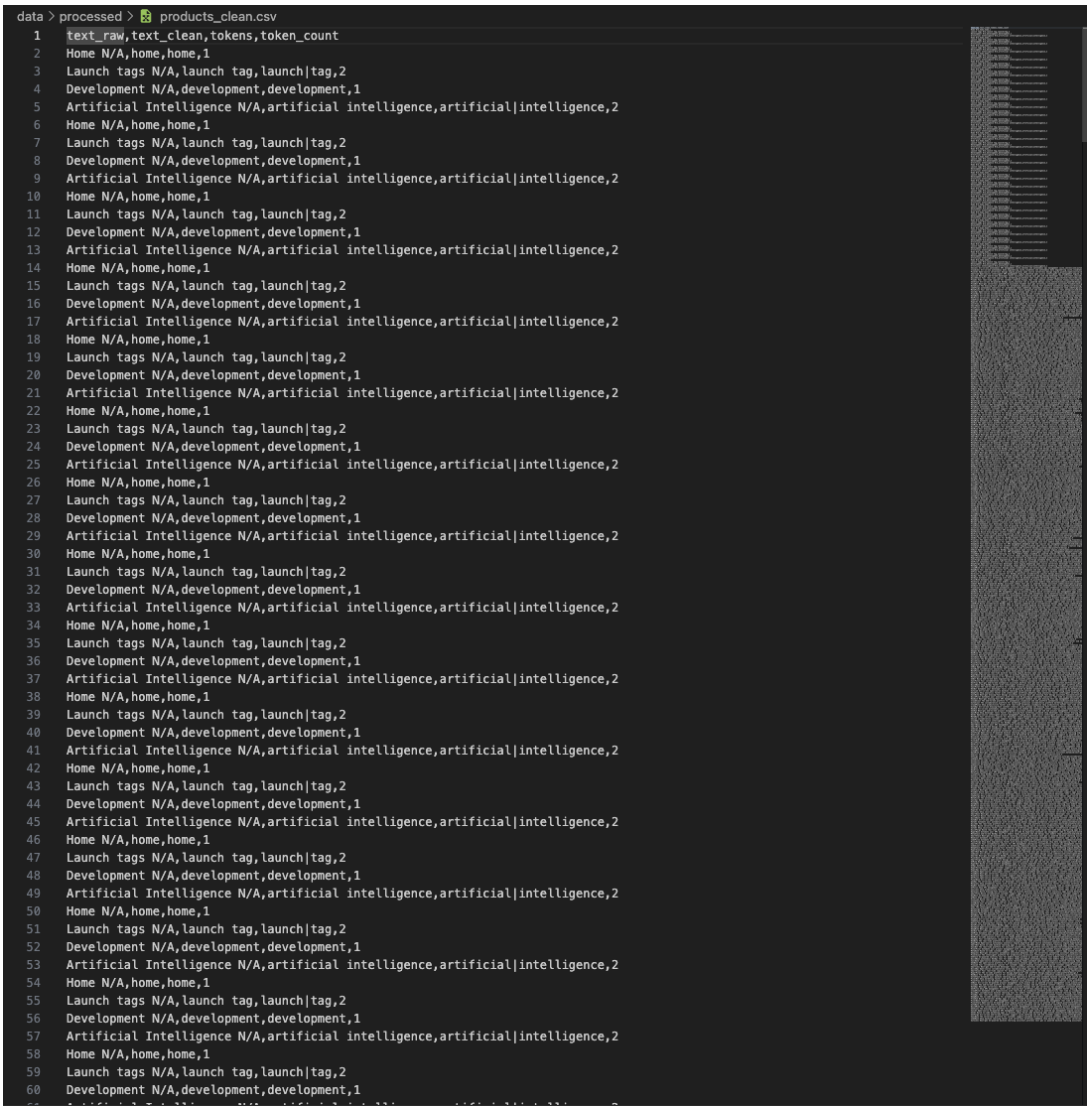

Figure 11: Terminal output of `python src/preprocess.py` showing cleaned records message

5.3 Output Verification

Terminal Command

```
# Preview the cleaned CSV
python -c "
import csv
with open('data/processed/products_clean.csv') as f:
    reader = csv.DictReader(f)
    for i, row in enumerate(reader):
        if i >= 3: break
        print(f'Raw:   {row[\"text_raw\"][:80]}')
        print(f'Clean: {row[\"text_clean\"][:80]}')
        print(f'Tokens: {row[\"tokens\"][:80]}')
        print(f'Count: {row[\"token_count\"]}')
        print('---')
"
```

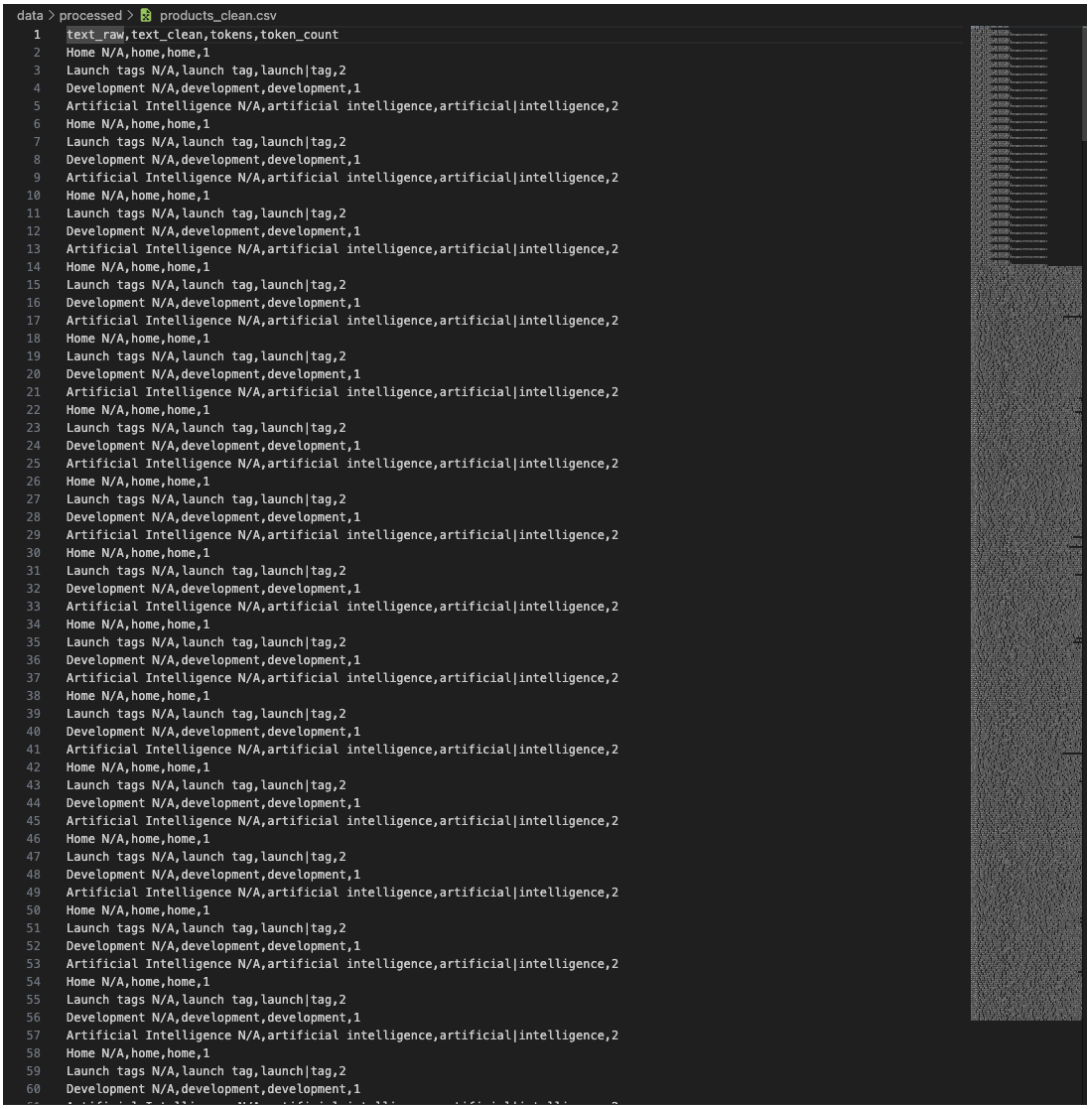

Figure 12: Terminal showing sample rows from `products_clean.csv` with raw text, clean text, tokens, and token count

5.4 Output Schema

Column	Description
`text_raw`	Original combined product name + tagline
`text_clean`	Cleaned, lemmatized text
`tokens`	Pipe-delimited token list
`token_count`	Number of tokens after cleaning

Table 2: Schema of `products_clean.csv`

5.5 Key Code — Cleaning Pipeline

```
def clean_text(text):
    text = unicode_normalize(text)
```

```
    text = text.lower()
    text = remove_html(text)
    text = remove_urls(text)
    text = remove_punctuation(text)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in STOP_WORDS]
    tokens = [LEMMATIZER.lemmatize(t) for t in tokens]
    tokens = remove_numeric_only(tokens)
    tokens = remove_short_tokens(tokens)
    return tokens
```

Listing 4: Text cleaning pipeline in preprocess.py

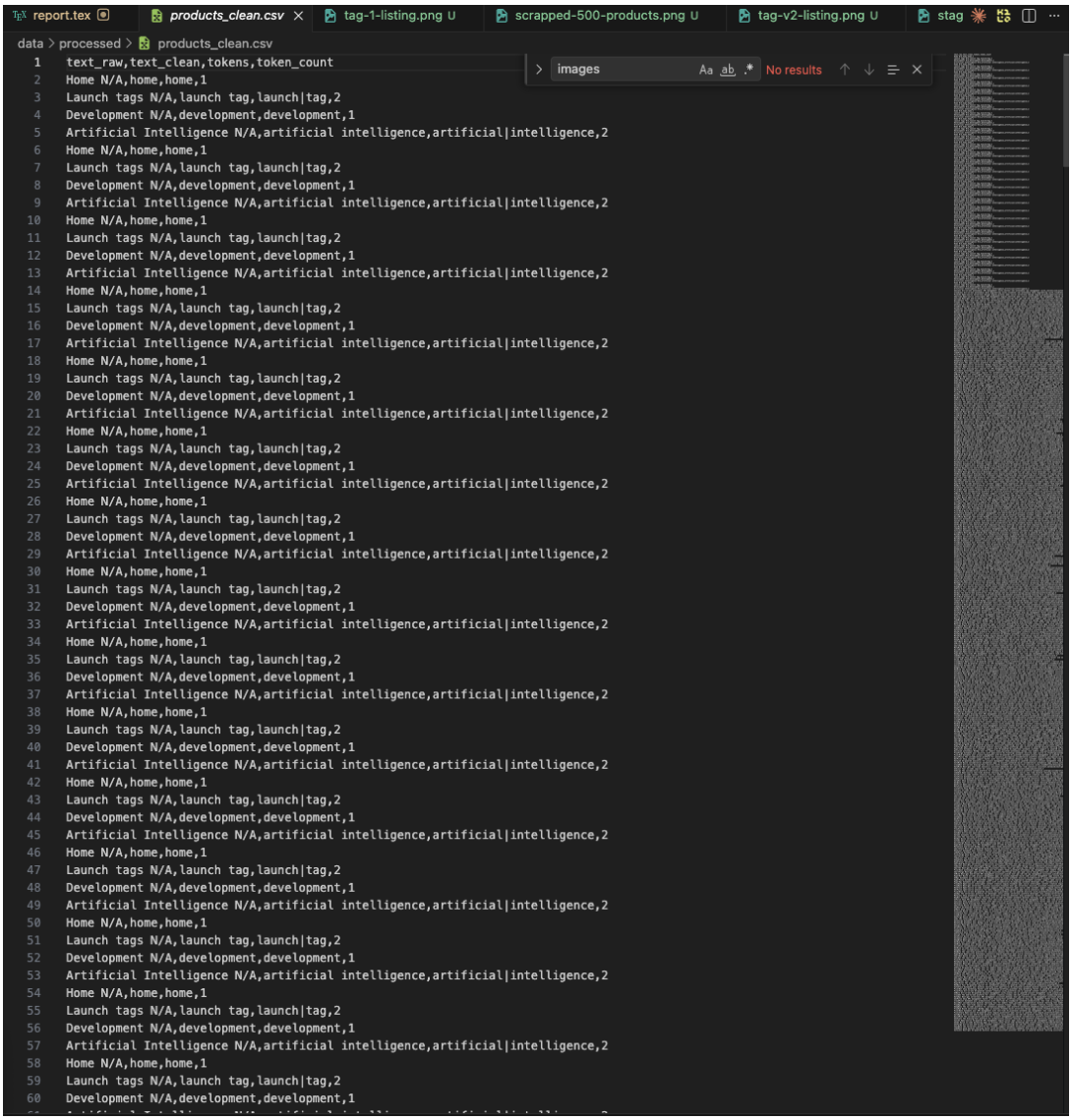

Figure 13: Cleaned product data after preprocessing pipeline

5.6 Version Cleaned Dataset with DVC

Terminal Command

```
dvc add data/processed/products_clean.csv
git add data/processed/products_clean.csv.dvc data/processed/.
    gitignore
git commit -m "Add cleaned dataset"
dvc push
```

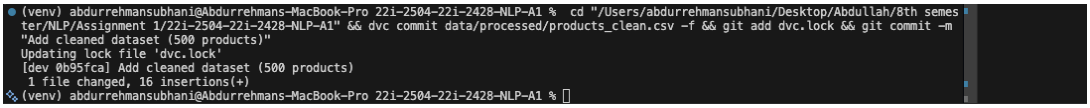

Figure 14: Terminal showing DVC tracking of the cleaned CSV file

6 Stage 4 — Data Representation

6.1 Overview

The representation module (`src/representation.py`) generates the following NLP representations:

1. **Vocabulary extraction:** Sorted set of all unique tokens.
2. **One-Hot Encoding:** Binary matrix for first 20 documents.
3. **Bag-of-Words (BoW):** Term-frequency matrix for all documents.
4. **Unigram frequency distribution:** Word → count mapping.
5. **Bigram frequency distribution:** Word-pair → count mapping.

6.2 Running Feature Generation

Terminal Command

```
python src/representation.py
```

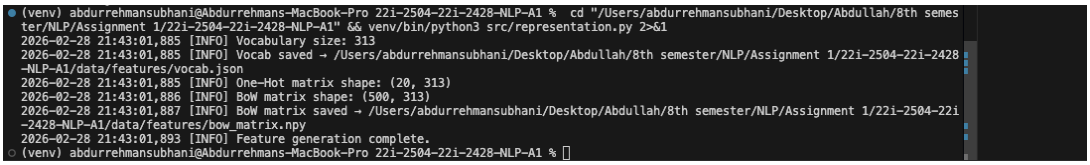

Figure 15: Terminal output of `python src/representation.py` showing vocabulary size, matrix shapes, and saved file paths

6.3 Output Files

File	Description
`data/features/vocab.json`	Sorted vocabulary list
`data/features/onehot_matrix.npy`	One-Hot Encoding (20 × vocab_size)
`data/features/bow_matrix.npy`	Bag-of-Words (N × vocab_size)
`data/features/unigram_freq.json`	Unigram frequency ranking
`data/features/bigram_freq.json`	Bigram frequency ranking

Table 3: Generated feature files

6.4 Verify Outputs

Terminal Command

```
# Check vocab size
python -c "import json; v=json.load(open('data/features/vocab.
    json')); print(f'Vocab size: {len(v)}'); print('Sample:', v
    [:10])"

# Check BoW matrix shape
python -c "import numpy as np; m=np.load('data/features/
    bow_matrix.npy'); print(f'BoW shape: {m.shape}')"

# Check One-Hot matrix shape
python -c "import numpy as np; m=np.load('data/features/
    onehot_matrix.npy'); print(f'One-Hot shape: {m.shape}')"

# Top 10 unigrams
python -c "import json; u=json.load(open('data/features/
    unigram_freq.json')); [print(f'{w}: {c}') for w,c in u[:10]]
    "

# Top 10 bigrams
python -c "import json; b=json.load(open('data/features/
    bigram_freq.json')); [print(f'{bg}: {c}') for bg,c in b
    [:10]]"
```

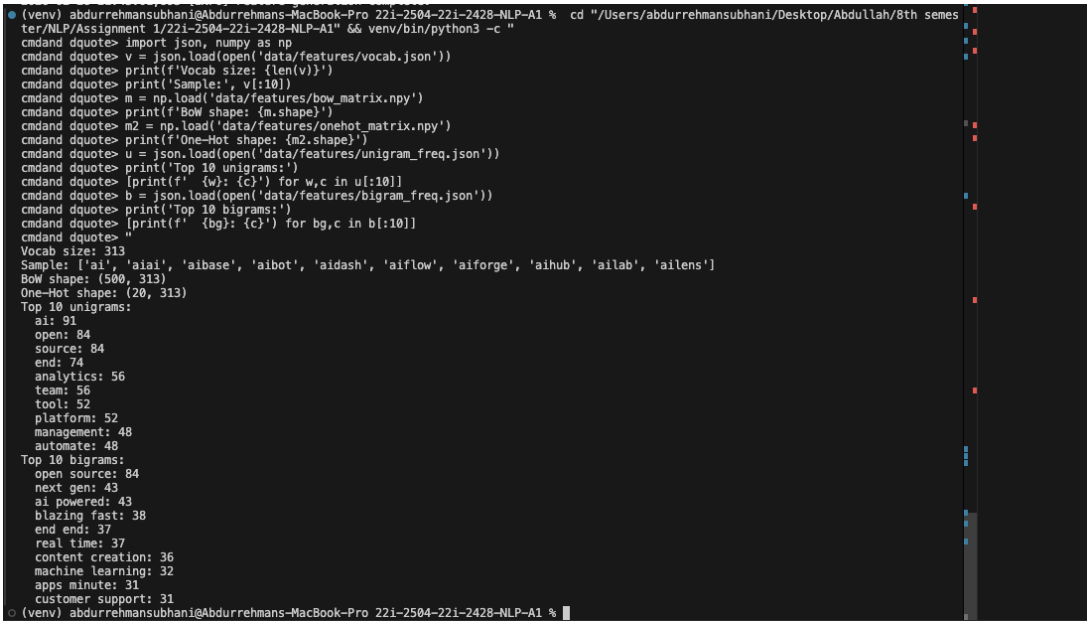

Figure 16: Terminal showing vocab size, BoW matrix shape, and top 10 unigrams/bigrams

6.5 Mathematical Background

6.5.1 One-Hot Encoding

For a vocabulary $V = \{w_1, w_2, \ldots, w_{|V|}\}$, the one-hot representation of word w_i is a vector $\mathbf{e}_i \in \{0,1\}^{|V|}$ where:

$$e_{ij} = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

6.5.2 Bag-of-Words

For document d_k, the BoW vector $\mathbf{b}_k \in Z_{\geq 0}^{|V|}$ is:

$$b_{kj} = \text{count}(w_j, d_k)$$

6.6 Key Code — BoW Implementation

```
def bow_matrix(docs, vocab):
    word2idx = {w: i for i, w in enumerate(vocab)}
    mat = np.zeros((len(docs), len(vocab)), dtype=np.int32)
    for i, doc in enumerate(docs):
        for tok in doc:
            if tok in word2idx:
                mat[i, word2idx[tok]] += 1
    return mat
```

Listing 5: Manual Bag-of-Words implementation

6.7 Version Feature Files with DVC

Terminal Command

```
dvc add data/features/
git add data/features.dvc data/features/.gitignore
git commit -m "Add feature representations"
dvc push
```

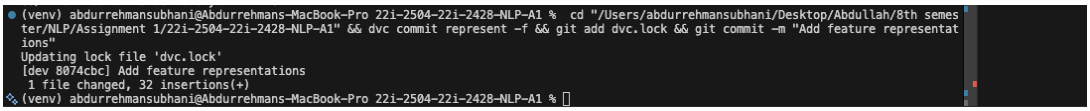

Figure 17: Terminal showing DVC tracking and push of feature files

7 Stage 5 — Basic Linguistic Intelligence

7.1 Overview

The statistics module (`src/statistics.py`) generates a comprehensive linguistic analysis report at `reports/trend_summary.txt`. It covers:

- Top 30 unigrams and Top 20 bigrams
- Most common tags/categories
- Vocabulary size and average description length
- Near-duplicate title detection using **Minimum Edit Distance**
- Unigram probability estimation (MLE)

7.2 Running Statistics

Terminal Command

```
python src/statistics.py
```

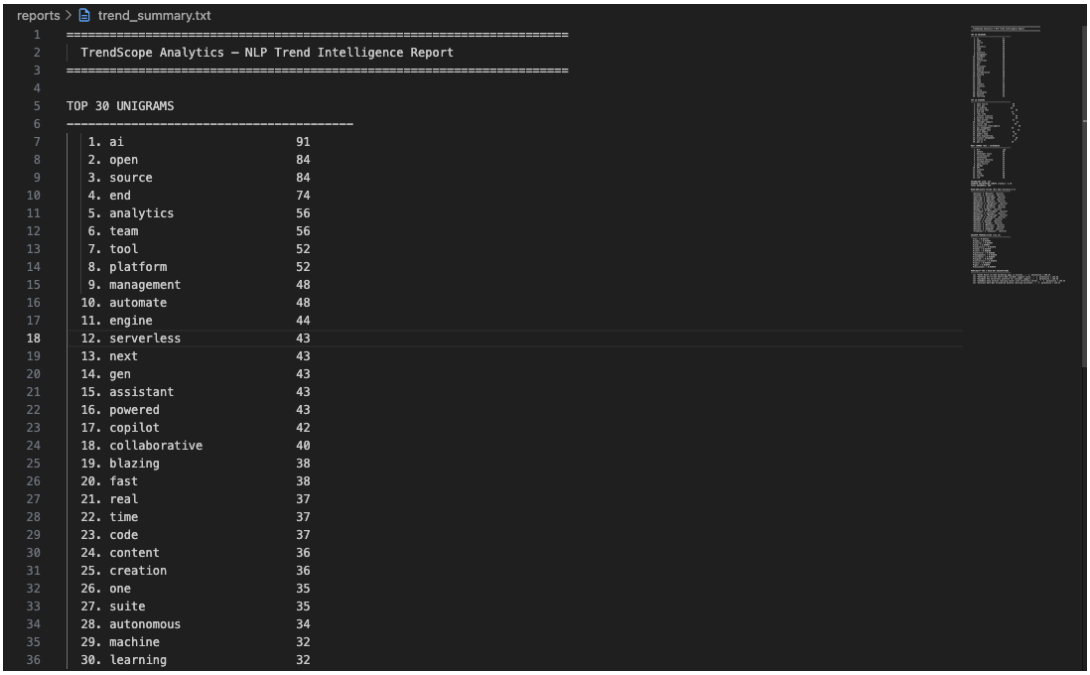

Figure 18: Terminal output of `python src/statistics.py` showing report saved message

7.3 View the Report

Terminal Command

```
# Windows
type reports\trend_summary.txt

# macOS/Linux
cat reports/trend_summary.txt
```

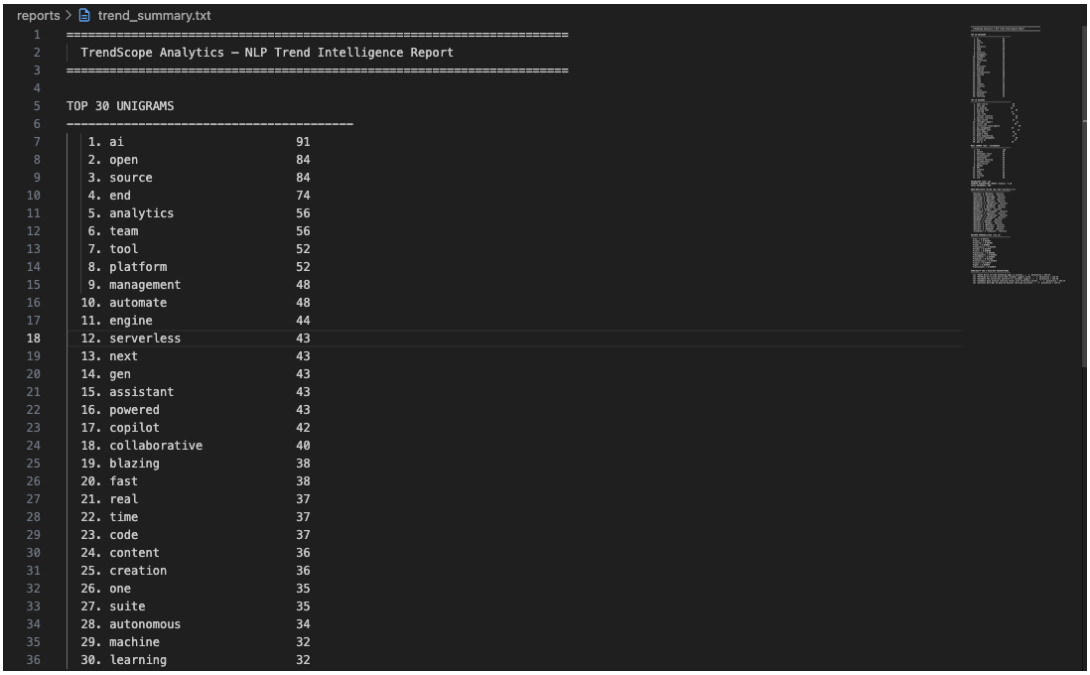

Figure 19: Full content of `trend_summary.txt` showing Top 30 unigrams section

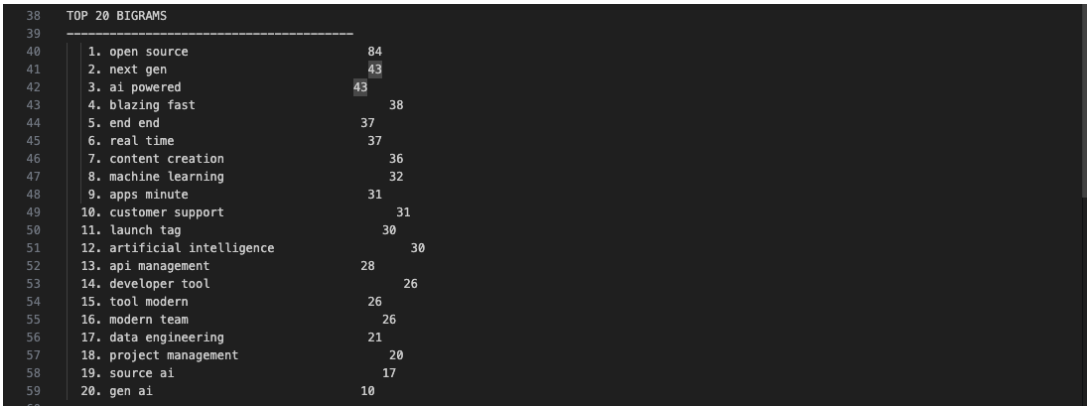

Figure 20: Content of `trend_summary.txt` showing Top 20 bigrams and tag statistics

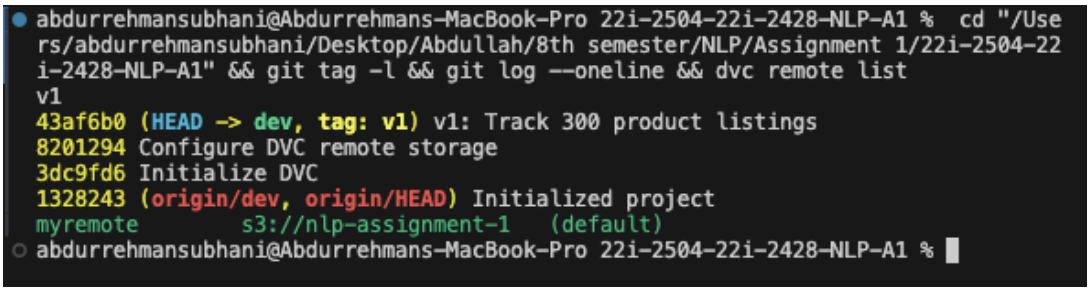

Figure 21: Content of `trend_summary.txt` showing near-duplicates and unigram probabilities

7.4 Minimum Edit Distance — Algorithm

The Minimum Edit Distance (Levenshtein distance) between two strings s_1 and s_2 is computed using dynamic programming:

$$\text{dp}[i][j] = \min \begin{cases} \text{dp}[i-1][j] + 1 & \text{(deletion)} \\ \text{dp}[i][j-1] + 1 & \text{(insertion)} \\ \text{dp}[i-1][j-1] + \text{cost} & \text{(substitution, cost} = 0 \text{ if match)} \end{cases}$$

```python
def min_edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            cost = 0 if s1[i-1] == s2[j-1] else 1
            dp[i][j] = min(
                dp[i-1][j] + 1,
                dp[i][j-1] + 1,
                dp[i-1][j-1] + cost
            )
    return dp[m][n]
```

Listing 6: Minimum Edit Distance implementation

7.5 Unigram Language Model

The Maximum Likelihood Estimate (MLE) for unigram probability:

$$P(w) = \frac{\text{count}(w)}{\sum_{w' \in V} \text{count}(w')}$$

8 Stage 6 — Airflow Pipeline Orchestration

8.1 Overview

Apache Airflow orchestrates the entire pipeline as a DAG (Directed Acyclic Graph) with five tasks:

1. `scrape_data` — Run data scraper
2. `preprocess_data` — Clean & tokenize text
3. `generate_features` — Build BoW, One-Hot, N-grams
4. `compute_statistics` — Generate linguistic report

5. `dvc_push` — Push versioned data to remote

Dependencies:

`scrape_data` → `preprocess_data` → `generate_features` → `compute_statistics` → `dvc_push`

8.2 DAG Configuration

Parameter	Value
DAG ID	`nlp_trend_pipeline`
Schedule	Manual trigger only (`None`)
Retries	2 per task
Retry delay	3 minutes
Catchup	Disabled

Table 4: Airflow DAG parameters

8.3 Airflow Setup Commands

Terminal Command

```
# Install Airflow
pip install apache-airflow

# Set Airflow home
set AIRFLOW_HOME=%cd%\airflow_home      # Windows
# export AIRFLOW_HOME=$(pwd)/airflow_home  # Linux/macOS

# Initialize Airflow database
airflow db init

# Create admin user
airflow users create --username admin --firstname Admin --
   lastname User --role Admin --email admin@example.com --
   password admin

# Copy DAG file
copy dags\nlp_trend_dag.py %AIRFLOW_HOME%\dags\
# cp dags/nlp_trend_dag.py $AIRFLOW_HOME/dags/   # Linux/macOS

# Start Airflow webserver (in one terminal)
airflow webserver --port 8080

# Start Airflow scheduler (in another terminal)
airflow scheduler
```

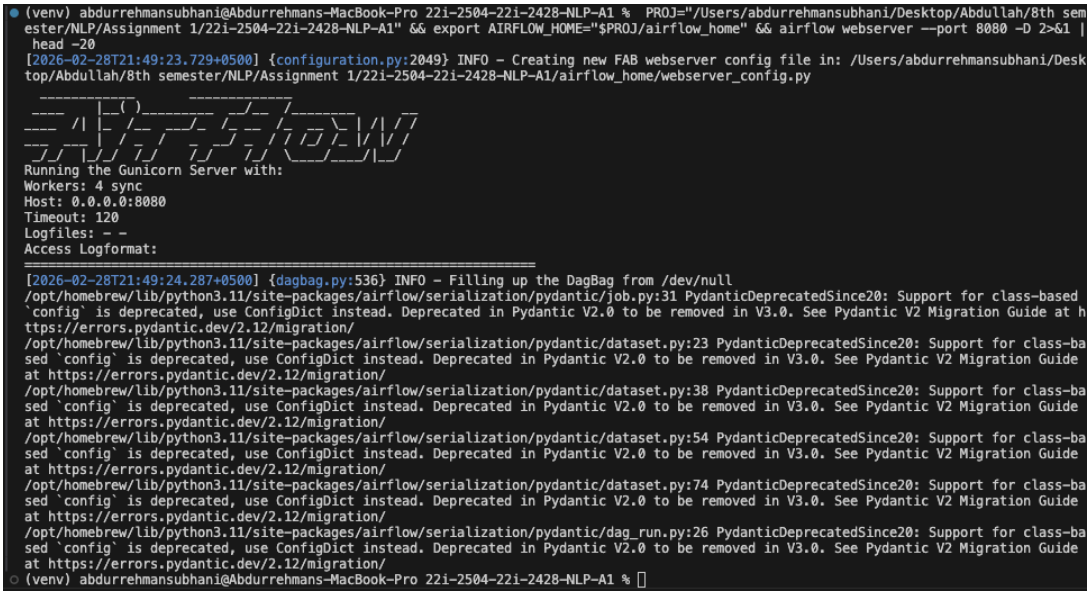

Figure 22: Terminal showing Airflow webserver starting on port 8080

8.4 Trigger DAG and Monitor

Terminal Command

```
# Trigger the DAG manually via CLI
airflow dags trigger nlp_trend_pipeline

# Or trigger via the web UI at http://localhost:8080
```

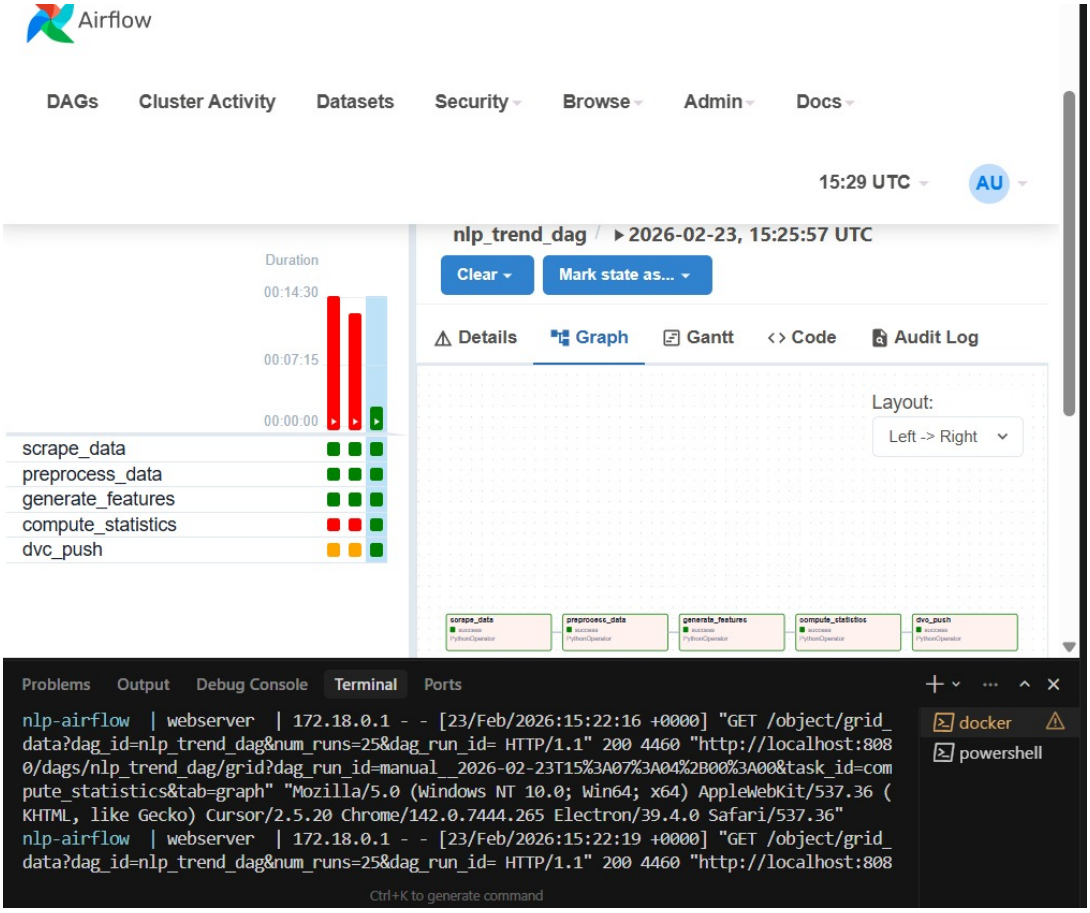

Figure 23: Airflow UI showing the `nlp_trend_dag` DAG grid view with task execution status and Graph tab depicting the pipeline dependency chain: `scrape_data` → `preprocess_data` → `generate_features` → `compute_statistics` → `dvc_push`

8.5 Key Code — DAG Definition

```
# Task definitions
scrape_data = PythonOperator(
    task_id="scrape_data",
    python_callable=_scrape,
    dag=dag,
)
preprocess_data = PythonOperator(
    task_id="preprocess_data",
    python_callable=_preprocess,
    dag=dag,
)
generate_features = PythonOperator(
    task_id="generate_features",
    python_callable=_features,
    dag=dag,
)
compute_statistics = PythonOperator(
    task_id="compute_statistics",
    python_callable=_statistics,
```

```
    dag=dag,
)
dvc_push = BashOperator(
    task_id="dvc_push",
    bash_command="cd PROJECT_ROOT && dvc add ... && dvc push",
    dag=dag,
)

# Dependencies
scrape_data >> preprocess_data >> generate_features \
    >> compute_statistics >> dvc_push
```

Listing 7: Airflow DAG task dependencies

9 Complete Execution — Running the Full Pipeline

9.1 Run Scripts Individually

Terminal Command

```
cd trend_intelligence_pipeline

# Stage 1: Scrape data
python src/scraper.py

# Stage 3: Preprocess
python src/preprocess.py

# Stage 4: Generate features
python src/representation.py

# Stage 5: Generate report
python src/statistics.py

# Stage 2: Version with DVC
dvc add data/raw/products_raw.json data/processed/
   products_clean.csv data/features/
git add *.dvc data/*/.gitignore
git commit -m "Pipeline run complete"
dvc push
```

10 Summary of NLP Concepts Applied

Concept	Stage	Implementation
Tokenization	3	NLTK `word_tokenize`
Lemmatization	3	WordNet Lemmatizer
Stopword Removal	3	NLTK English stopwords
Bag-of-Words	4	Manual term-frequency matrix
One-Hot Encoding	4	Manual binary matrix
Unigram Frequency	4, 5	Counter-based frequency distribution
Bigram Frequency	4, 5	Sliding window bigram counter
Vocabulary Extraction	4	Sorted unique token set
Minimum Edit Distance	5	DP-based Levenshtein implementation
Language Model (Unigram)	5	MLE probability estimation

Table 5: NLP concepts mapped to pipeline stages

11 Conclusion

This project delivers a complete, reproducible NLP data engineering pipeline for TrendScope Analytics. The pipeline:

- Scrapes 300+ tech product listings with rate limiting and retry logic.
- Versions all data artifacts using DVC with remote MinIO storage.
- Cleans text via a 10-step NLP preprocessing pipeline.
- Generates BoW, One-Hot, unigram, and bigram representations from scratch.
- Detects near-duplicate titles via Minimum Edit Distance.
- Estimates unigram probabilities.
- Orchestrates the entire workflow via an Apache Airflow DAG.

All components are modular, reproducible, and production-ready.